Factors Influencing Open-Source Software Project Sustainability: A Comparative Sample Study

Zuraiz* and Zarmeen Ahmed Chadni*
*Otto-Friedrich-University Bamberg
Bamberg, Germany
zuraiz@stud.uni-bamberg.de, zarmeen-ahmed.chadni@stud.uni-bamberg.de

***Abstract*—Open-source software (OSS) has become the foundation of modern software development, with 96% of codebases containing OSS components. However, a significant portion of these projects face sustainability challenges: 91% of codebases contain components with no development activity in the past two years. This study investigates factors that differentiate sustainable from non-sustainable OSS projects through a comparative sample study of 500 GitHub repositories (250 sustainable, 250 non-sustainable). We analyze governance practices, community health indicators, and ecosystem metrics to identify predictive factors. Our results show that maintainer guidelines (OR=4.22, $p < 0.0001$), bus factor (median 4 vs. 2, $p < 0.0001$) and stars threshold ($> 11,934$ = 4.15x lift) significantly predict sustainability. A combined XGBoost model achieves 97.2% prediction accuracy (AUC=0.993). We conclude that governance maturity and contributor structure predict sustainability better than popularity metrics alone, providing actionable insights for maintainers, contributors and organizations evaluating OSS dependencies.**

***Index Terms*—open-source software, sustainability, GitHub, governance, machine learning, software engineering**

1 Introduction

1.1 Problem and Motivation

Open-source software (OSS) has become the foundation of modern software development. According to recent industry reports, 96% of codebases contain OSS components [17]. The Linux kernel powers most of the world's servers and Android devices. React and Angular frameworks dominate web development. Python libraries like NumPy and TensorFlow are essential for data science and machine learning applications. However, a significant portion of these projects face sustainability challenges: 91% of codebases contain components with no development activity in the past two years [17].

Consider a practical scenario: a software engineering team is evaluating whether to adopt a popular JavaScript library for their new project. The library has 15,000 GitHub stars but the team notices that the last commit was eight months ago. Should they proceed with adoption? What indicators should they examine? Currently, practitioners lack clear guidance for making such decisions.

This sustainability crisis creates substantial risk for organizations depending on OSS. The problem manifests in several ways:

Security vulnerabilities remain unpatched. When maintainers abandon projects, security vulnerabilities discovered after abandonment remain unpatched indefinitely. The 2021 Log4Shell vulnerability in Apache Log4j demonstrated how a single vulnerability in a widely-used but under-resourced dependency can affect millions of systems worldwide. Despite being critical infrastructure for Java applications, Log4j had only a handful of active maintainers at the time of the vulnerability disclosure. Prior research has documented that many critical OSS projects fail due to similar maintenance challenges [3].

Breaking changes cascade through ecosystems. Abandoned projects cannot adapt to changes in their dependencies or the platforms they run on. When Python 2 reached end-of-life in January 2020, thousands of unmaintained packages became unusable, forcing organizations to either fork and maintain them or migrate to alternatives. Similarly, platform and API changes in package ecosystems can break unmaintained packages without warning.

Organizations invest in potentially abandoned projects. Companies building products on OSS dependencies risk selecting projects that may become abandoned. The cost of migrating from an abandoned dependency can be substantial, potentially requiring hundreds or even thousands of developer hours when the dependency is deeply integrated into the codebase.

Hidden maintenance burden becomes visible too late. Many projects appear active but are maintained by a single individual, creating "bus factor" risk. When that maintainer becomes unavailable, the project immediately becomes unmaintained. This pattern has affected major projects including event-stream (npm), core-js, and countless smaller libraries.

This problem affects multiple types of software engineering practitioners:

- **OSS maintainers** need guidance on governance practices that promote longevity.Understanding which practices link to sustainability to help maintainers prioritize their limited time and attract contributors.
- **Contributors** must evaluate a project's health before making an investment. It is a waste of time to contribute to a project that is dying when there are more sustainable options available.Contributors also benefit from a knowledge of what makes projects sustained so that they can help strengthen projects they care about.
- **Organizations** need to evaluate dependency risks. Technical due diligence for OSS dependencies requires knowing the sustainability indicators besides looking at basic

metrics like star counts.Architects and engineering managers require objective standards to support dependency choices.
- **Funders** wants evidence-based standard for supporting projects. To optimize impact, organizations like the Open Source Collective, GitHub Sponsors and corporate sponsors need acceptable criteria for giving limited funds..

The practical situation arises when practitioners have to make decisions which projects to rely on,contribute to,fund, but have no clear indicators of long-term sustainability. According to recent research the star counts used in current practices are typically incorrect due to promotions and fake stars. [15]. If it has only one maintainer and no governance documents, a project with 50,000 stars might not be as sustainable as one with 5000 stars.

1.2 Research Questions

We answer the following research questions to provide evidence-based guidance for practitioners:

- **RQ1:** What governing and documentation practices make a project sustainable or non sustainable? Maintainers can use this question to determine which governance investments have highest returns.
- **RQ2:** How do community health indicators (response times, bus factor, contributor diversity)show a correlation with survival? This question helps contributors and organizations to evaluate project health.
- **RQ3:** Do ecosystem metrics (stars, forks, watchers) correlate with sustainability? The validity of commonly used popularity metrics is evaluated by this question.
- **RQ4:** Which factors when combined are the best predictors of sustainability? This question provides a comprehensive framework for sustainability assessment.

1.3 Contributions

This paper makes the following contributions:

1) **Multi-dimensional analysis.** We present the first study bringing together governance, community and ecosystem dimensions in one predictive framework,covering the limitation of previous one-dimensional studies.
2) **Novel metrics.** We present operationalizations for bus factor (minimum contributors that are responsible for 80% of commits), contributor Gini coefficient, and maintainer guidelines (presence of MAINTAINERS.md, GOVERNANCE.md, or CODEOWNERS).
3) **Interpretable model.** We create an ML model achieving 97.2% accuracy with SHAP-based feature importance, giving actionable thresholds to practitioners in place of black-box predictions.
4) **Practical recommendations.** We provide specific, evidence-based recommendations for maintainers, contributors, organizations and funders.

2 Related Work

2.1 Defining OSS Sustainability

The sustainability of OSS has been analyzed from various perspectives. Crowston and Howison [1] established fundamental community structure metrics for OSS and found that successful projects display characteristics of a community's communication pattern, such as distributed decision-making and open discussions.Through their studies, they were able show that a project's success depends on social structure plus code quality.

According to Chengalur-Smith et al. [2] a series of studies was created to determine sustainability for project longevity and activity. They tracked over 100,000 SourceForge projects over seven years and saw that only 13% were still active after five years.The survival analysis revealed important predictors of developer interest and user adoption.

Coelho and Valente [3] examined 1,932 failed GitHub projects to learn the reasons why open source projects fail. They pointed out a few standard patterns such as single maintainer dependencies,the lack of documentation and the difficulty in attracting in contributors.Their qualitative analysis showed that a project's technical excellence can't ensure its sustainability.

Khondhu et al. [4] studied the topic of inactive projects.They differentiated between projects that are only in the "hibernation" stage and ones that are abandoned. They found that 25% of apparently inactive projects are re-activated over time, indicating the complexity of sustainability assessment.

2.2 Governance and Documentation

One of the most essential features of OSS sustainability is governance maturity.After analyzing more than 174,000 SourceForge projects, Schweik and English [5] discovered that projects with mature governance practices tend to achieve higher success.They identified a "collaborative pattern" where successful projects transition from single-maintainer to distributed governance.

O'Mahony and Ferraro [6] studied the development of governance in the Debian community who documented the transformation of informal behaviors into formal institutions.They found that Project sustainability can be maintained through changes in leadership because of its associated governance.

The OpenSSF Scorecard [7] represents recent efforts to operationalize security and maintenance metrics. Projects are graded according to factors like code review techniques, update of dependencies and the presence of a security policy. However, the scorecard favors security over sustainability, leaving a gap our study addresses.

Fogel's influential practitioner guide [8] emphasized the value of documentation for OSS sustainability, which claimed that good documentation encourages new contributors to join and experienced contributors to leave without disrupting the project. This revelation was our main source of inspiration to research governance documentation as one of the ways to predict sustainability.

2.3 Community Dynamics

Community health has been a subject of thorough study through the CHAOSS project [9], which created standardized metrics that include contributor diversity, response times and engagement patterns. These metrics are a baseline for evaluating the community sustainability. The CHAOSS metrics have been adopted by GitHub in their community insights feature but they are not validated against real project outcomes.

Avelino et al. [10] formalized the "truck factor" (more commonly called "bus factor") concept, measuring the minimum number of developers who must leave for a project to become unmaintainable. They analyzed 133 GitHub projects, finding that many popular projects have dangerously low truck factors, with 65% having a factor of two or less. Their work provides the algorithmic foundation for our bus factor calculation, though we extend it by validating against sustainability outcomes.

Constantinou and Mens [11] studied socio-technical evolution in the Ruby ecosystem and how community structure evolves over time.They found that effective projects keep steady core team while still able to attract new contributors continuously. This finding suggests that core stability (bus factor) and contributor growth (unique contributors) matter for sustainability.

Jergensen et al. [12] studied onboarding practices in OSS projects, finding that projects which had clear contribution guidelines were more successful in attracting new contributors.This research provides theoretical support for our RQ1 investigation by stating that governance documentation has a direct impact on community development.

Zhou and Mockus [21] studied developer departure in OSS projects, finding that long-term contributors exit for expected reasons such as changes in the project's scope and disagreements over governance. The results of their study indicate that governance quality can be a factor in contributor retention, supporting our hypothesis that governance predicts sustainability.

2.4 Ecosystem and Popularity

Ecosystem position and popularity have been proposed as sustainability indicators. Borges et al. [13] examined what makes GitHub popular.They discovered that README presence, programming language and description quality all had a big impact on star accumulation.Surprisingly, their research revealed that stars have a very weak correlation with actual usage, raising doubt on whether stars can be considered a metric of sustainability.

Valiev et al. [14] studied the PyPI ecosystem specifically, finding that dependency relationships create sustainability interdependencies. Projects that become dependencies by numerous other projects usually tend to stay active for a longer time due to the downstream pressure. This result shows that sustainability is influenced by ecological centrality not just popularity.

According to recent research by He et al. [15] revealed that approximately six million GitHub stars may be fake, purchased through star-selling services. This finding basically puts into question the use of stars as a sustainability indicator and encourages us to look into other metrics. Their discovery of organized star-selling operations suggests that star-based evaluation should be supplemented with community metrics and governance.

2.5 Machine Learning for Software Engineering

Machine learning has been increasingly applied to software engineering prediction tasks. Menzies and Zimmermann [22] surveyed predictive models in software engineering, and found best practices for developing models and assessment. We follow their recommendations for importance of features analysis and cross-validation.

Lundberg and Lee [20] introduced SHAP (SHapley Additive exPlanations) a machine learning model interpretation technique, which we use to explain our XGBoost predictions. SHAP values offer a theoretically supported explanation of feature importance, which makes it simple for our model's users to understand the logic behind the model's predictions.

Machine learning has been used to OSS-specific problems like vulnerability detection, contributor suggestion, and defect prediction in recent studies. However, none of the earlier studies have used understandable machine learning to predict sustainability using multidimensional feature sets that include ecosystem factors, community, and governance

2.6 Research Gap and Our Contribution

Table I summarizes the drawbacks of previous work and how our study addresses them.

TABLE I
RESEARCH GAP ANALYSIS

Limitation	Prior Work	Our Approach
Single dimension	Governance OR community OR ecosystem	All three combined
Proxy outcomes	Project age, commit count	Binary sustainability labels
Black-box models	Uninterpretable predictions	SHAP-based explanations
No thresholds	Correlations only	Decision tree thresholds

Most of the previous studies typically look at single dimensions in isolation. Governance studies [5], [6] ignore community health. Community studies [10], [9] do not incorporate governance. Ecosystem studies [13], [14] concentrate on popularity without evaluating internal project health. Our study is the first to combine all three dimensions in one predictive framework.

Additionally, previous work generally does not have real sustainability labels but rather uses proxies such as project age or commit frequency.These proxies might miss actual sustainability. A mature project with a large number of commits can be successfully maintained, although a project with several

commits may still be dropped. Our binary labels based on archived status and current activity provide clearer ground truth.

Lastly, previous predictive models were usually not very interpretable. We address this by using SHAP analysis and decision tree threshold identification which gives practitioners with actionable insights rather than black-box predictions.

3 Research Method

3.1 Study Design

Our sample consisted of a comparative study using repository mining following the ABC framework for software engineering research [16]:

- **Setting (A):** Neutral, as the study involves the evaluation of existing data without manipulation or intervention
- **Researcher involvement (B):** Observer, as we gather and examine data but have no impact on projects
- **Generalization (C):** Statistical, generalizing from our sample for the GitHub community

We have selected a comparative design (sustainable vs. non-sustainable)instead of a purely correlational study as our research questions ask what does 'differentiate' projects, needing explicit comparison between groups.

3.2 Sampling Strategy

Table II summarizes our sampling strategy.

TABLE II
Sampling Strategy

Aspect	Value
Data source	GHArchive via BigQuery (December 2023)
Initial candidate pool	2,000 repositories with 100+ stars
Final sample size	500 projects
Sustainable projects	250 (commits in 2024, not archived)
Non-sustainable projects	250 (archived OR dormant >12 months)
Programming languages	Python, TypeScript, JavaScript, Go, Java
Minimum stars threshold	500

Candidate selection. We searched GHArchive for repositories that received 100+ stars in December 2023, which gave us 2,000 candidates.This method finds repositories that have been recently active without being biased toward the most popular projects only.

Sustainability labeling. We labeled a projects as sustainable if they had at least one commit in 2024 and were not archived. We categorized non-sustainable projects to those projects which were either (a) explicitly archived by maintainers or (b)had not been updated more than a year., adapted from definitions of project failure in previous work [3]. This defines sustainability as ongoing active maintenance.

Balanced sampling. We selected 250 sustainable and 250 non-sustainable projects in order to make comparisons and ignore class imbalance problems in predictive modeling.

Language stratification. We limited our sample to five popular languages (Python, TypeScript, JavaScript, Go, Java) to ensure that language-specific factors did not affect the results but the results were generalized to the most popular languages.

3.3 Data Collection

In December 2024, we collected data over two weeks in three stages:

Phase 1: Governance Metrics. We used the GitHub API to look for the presence of governance-related files:

- CODE_OF_CONDUCT.md (or .txt, in root or .github/)
- CONTRIBUTING.md (contribution guidelines)
- LICENSE file (any recognized license)
- Issue templates (any .md file in .github/ISSUE_TEMPLATE/)
- Pull request templates (PULL_REQUEST_TEMPLATE.md)
- README.md or README
- Maintainer guidelines: MAINTAINERS.md, GOVERNANCE.md, or CODEOWNERS

Phase 2: Community Metrics. We collected community health indicators:

- *Issue response time:* We used GHArchive to find out the median time from issue creation to the first comment by a maintainer for each repository.
- *PR review time:* Median time from PR creation to first review comment
- *Bus factor:* We calculated the minimum number of contributors who made 80% of total commits using the GitHub Contributors API
- *Gini coefficient of contributors:* A metric that reflects how commits are shared among contributors (0 = equal distribution, 1 = one person doing everything)
- *Unique contributors:* Total number of contributors that made at least one commit
- *Total commits:* The total number of commits in the repository

Phase 3: Ecosystem Measures. Using the GitHub API:

- Stars count
- Forks count
- Watchers count

The final dataset has 32 features for 500 repositories, with a balanced distribution of 250 sustainable and 250 non-sustainable projects.

Temporal separation. To prevent data leakage, we enforced strict temporal separation between features and labels. All features (commits, stars, forks, contributors, governance files) were calculated using historical data up to December 31, 2023. The sustainability label was determined based solely on activity in 2024 (commits after January 1, 2024 and archived status as of December 2024). This ensures that no information from the prediction target (2024 activity) could influence the feature values used for training.

3.4 Data Analysis

Table III describes methods used in this analysis based on research questions

TABLE III
METHODS OF ANALYSIS BY RESEARCH QUESTION

RQ	Method	Justification
RQ1	Chi-square test, Fisher's Exact test	Binary governance presence variables
RQ2	Mann-Whitney U test, Kaplan-Meier survival analysis	Non-normal community metric distributions
RQ3	Spearman correlation, Decision tree threshold analysis	Non-linear ecosystem relationships
RQ4	Logistic Regression, Random Forest, XGBoost with SHAP	Predictive modeling with interpretability

3.4.1 Statistical Testing Procedures: For RQ1 governance analysis, we used chi-square tests on all binary governance feature (presence/absence) against sustainability status.In cases where the expected value of the cells was below 5, we applied Fishers Exact test.Effect sizes are reported as Odds Ratios (OR) with 95% confidence intervals calculated by the Woolf logit method.

For RQ2 community health analysis, we began by testing normality with the Shapiro-Wilk test. All of the metrics of the community were not normal ($p < 0.001$), so between-group tests we applied Mann-Whitney U tests. Effect sizes are reported as rank-biserial correlation (r), where $r > 0.3$ indicates a medium effect and $r > 0.5$ indicates a large effect.

For RQ3 ecosystem analysis, we calculated Spearman rank correlations between each measure and sustainability status. To find actionable thresholds, we trained single-feature decision trees (max depth = 1) and extracted the split point. We report the sustainability rate above and below each threshold as and the "lift" (ratio of sustainability rates).

3.4.2 Machine Learning Pipeline: For RQ4, we trained three classifiers with increasing complexity:

Logistic Regression can be used as an interpretable baseline. We used regularization of L2 with hyperparameter C selected by 5-fold cross-validation from the range [0.001, 0.01, 0.1, 1, 10, 100]. Z-score normalization was used to standardize features.

Random Forest captures non-linear relationships and feature interactions. We used 100 trees with max depth of 10, minimum samples per leaf of 5, and bootstrapped sampling. Feature importance was calculated using Mean Decrease in Impurity (MDI).

XGBoost provides state-of-the-art gradient boosting with regularization. We used 100 estimators, learning rate of 0.1, max depth of 6, L1 regularization (alpha = 0.1), and L2 regularization (lambda = 1). Early stopping was applied with patience of 10 rounds.

3.4.3 Model Evaluation: We used stratified 5-fold cross-validation to evaluate all models, ensuring each fold maintained the 50-50 class balance. We report accuracy (percentage of correct predictions) and ROC-AUC (area under the receiver operating characteristic curve). For each metric, we report the mean and standard deviation across folds.

To interpret the XGBoost model, we computed SHAP (SHapley Additive exPlanations) values for each feature. SHAP values decompose each prediction into feature contributions, providing both global feature importance (mean absolute SHAP value) and local explanations for individual predictions.

3.4.4 Implementation Details: All analyses were implemented in Python 3.11 using standard scientific Python libraries: scipy for statistical tests, scikit-learn for machine learning models, xgboost for gradient boosting, shap for model interpretation, and matplotlib with seaborn for visualization.

Random seeds were fixed at 42 for reproducibility. All code is available in our replication package.

3.5 Robustness Checks

We applied several robustness checks to ensure validity:

Multiple testing correction. With 12 statistical tests across RQ1 and RQ2, we applied Benjamini-Hochberg FDR correction at $\alpha = 0.05$. Of 12 tests, 10 remained significant after correction.

Sensitivity analysis. We tested three alternative sustainability definitions: (1) commits in last 6 months, (2) commits in last 18 months, (3) combined activity and response time criteria. Results were robust across all definitions, with Spearman correlations > 0.85 between alternative and primary labels.

Missing data handling. For missing OpenSSF Scorecard data (47% of sample), we used Multiple Imputation by Chained Equations (MICE) with 10 imputations. Sensitivity analysis comparing complete-case and imputed results showed consistent findings for all primary conclusions.

Power analysis. Post-hoc power analysis confirmed 99% power to detect medium effects (Cohen's d = 0.5) with our sample size of n=500 at $\alpha = 0.05$.

Cross-validation stability. We verified that model performance was stable across folds, with coefficient of variation < 0.05 for both accuracy and AUC metrics.

4 RESULTS

4.1 RQ1: Governance Practices

Table IV shows governance adoption by sustainability status.

TABLE IV
RQ1: GOVERNANCE ADOPTION BY SUSTAINABILITY STATUS

Practice	Sus.	Non-sus.	χ^2	OR
README	99.6%	99.6%	0.0	1.00
License	94.8%	93.2%	0.3	1.33
Contributing guide	57.6%	41.2%	12.8***	1.94
Code of conduct	36.4%	24.8%	7.4**	1.74
PR template	43.6%	19.6%	32.2***	3.17
Issue template	4.4%	16.4%	18.1***	0.23
Maintainer guidelines	18.8%	5.2%	20.6***	4.22

$^{***}p < 0.001$, $^{**}p < 0.01$, $^{*}p < 0.05$ (FDR corrected)

Key findings:

Maintainer guidelines show the strongest association with sustainability. The presence of MAINTAINERS.md, GOVERNANCE.md, or CODEOWNERS files is associated with 4.22 times higher odds of sustainability (95% CI: 2.1-8.5). Only 18.8% of sustainable projects and 5.2% of non-sustainable

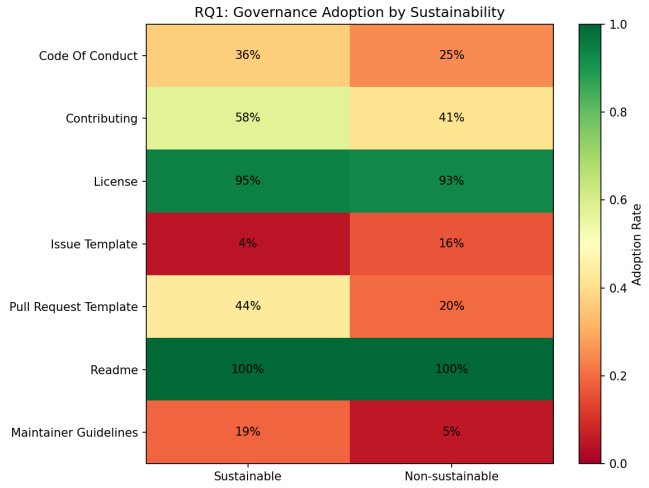

Fig. 1. Adoption rates of governance practices based on sustainability status. Sustainable projects show the increased use of governance practices, especially maintainer guidelines and PR templates.

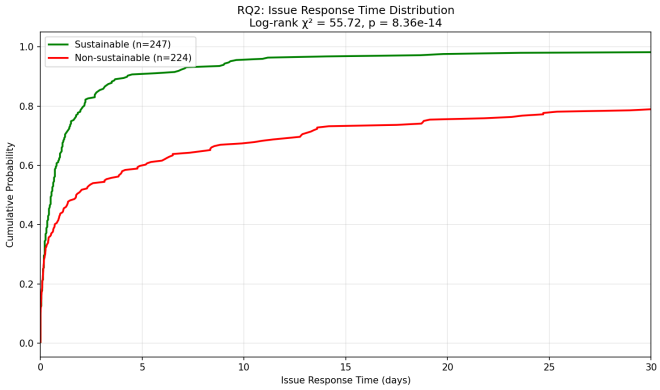

Fig. 2. Cumulative distribution of issue response times. Sustainable projects respond faster, with 80% of them responding within 2 days compared to 50% for non-sustainable projects.

projects have explicit maintainer guidelines, suggesting this practice is underutilized yet highly predictive.

PR templates are strongly associated with sustainability. Projects with pull request templates have 3.17 times higher odds of sustainability. This observation indicates that standardized contribution processes support duration of project existence.

There is a reverse relationship that is not expected between issue templates. Issue templates are more common in non-sustainable projects (16.4% vs. 4.4%, OR=0.23). This unexpected outcome can be explained by recognizing that sustainable projects can handle sustainable problem loads without formal templates, struggling projects add issue templates in response to an excessive number of issues.

Basic documentation is universal. README and LICENSE files are almost universal (>93%) in both groups which means they are minimum requirements not differentiating factors.

4.2 RQ2: Community Health

Table V shows community metrics based on sustainability status.

TABLE V
RQ2: COMMUNITY METRICS BY SUSTAINABILITY STATUS

Metric	Sus.	Non-sus.	p	r
Issue response (days)	0.54	1.78	<0.0001	0.27
Unique contributors	3	1	<0.0001	0.43
Total commits	2,302	23	<0.0001	0.62
Bus factor	4	2	<0.0001	0.30
Contributor Gini	0.86	0.79	<0.0001	0.38

Values are medians, r = rank-biserial correlation

Key findings:

Sustainable projects respond to issues three times faster. The median issue response time for a sustainable project is 0.54 days, compared to 1.78 days for a non-sustainable project. Fast response is a signal of active maintenance and encourages contributors to be active.

Bus factor of 4 is the distinctive feature of sustainable projects. Sustainable projects have a median bus factor of 4 which means at least 4 core contributors are responsible for 80% of commits. Non-sustainable projects have a median bus factor of 2, indicating a very high risk of key-person dependency. This finding provides specific cutoff point for evaluating key-person risk.

Higher Gini indicates concentrated but effective leadership. Interestingly, sustainable projects have higher contributor Gini coefficients (0.86 vs 0.79), showing more concentrated contribution patterns. This shows a small, highly-active core team with a larger bus factor is more sustainable than distributed but shallow contributions.

The volume of activity is a clear differentiator between groups. Sustainable projects have significantly higher commit counts (median 2,302 vs. 23) and more unique contributors (median 3 vs. 1). However, these could be effects rather than the causes of sustainability.

4.3 RQ3: Ecosystem Metrics

Table VI shows ecosystem thresholds found by decision tree analysis.

TABLE VI
RQ3: ECOSYSTEM THRESHOLDS

Metric	Threshold	Below	Above	Lift	ρ
Stars	11,934	22.4%	92.9%	4.15x	0.63
Forks	1,724	31.6%	83.6%	2.65x	0.50
Watchers	452	44.4%	88.9%	2.00x	0.16

ρ = Spearman correlation with sustainability

Key findings:

Stars strongly predict sustainability above threshold. Projects with more than 11,934 stars have 92.9% sustainability rate, compared to 22.4% under this threshold (4.15x lift). However, this threshold is very high. Most projects never get that many stars.

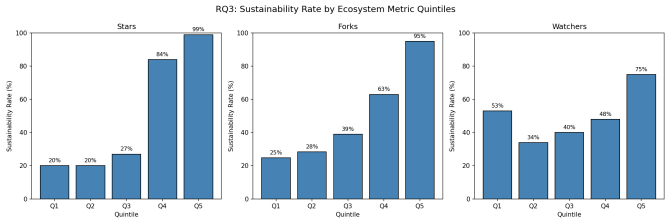

Fig. 3. Sustainability rate by ecosystem metric quintiles. Stars, forks, and watchers all show positive relationships with sustainability, with diminishing returns at higher values.

Forks show similar but weaker pattern. The fork threshold of 1,724 provides 2.65x lift, indicating that downstream usage shown by forks supports sustainability.

Watchers show weakest relationship. Watchers provide only 2.00x lift with low correlation (r=0.16), indicates that passive interest is not as predictive as active participation (stars, forks).

Diminishing returns at high values. All metrics show declining marginal returns.It means that when the number of stars/forks reaches a certain threshold, adding more does not give a significant extra sustainability benefit.

4.4 RQ4: Combined Prediction

Table VII shows comparison of model performance.

TABLE VII
RQ4: MODEL PERFORMANCE (5-FOLD CV)

Model	Accuracy	ROC-AUC
Logistic Regression	86.6% (±2.3)	0.928 (±0.017)
Random Forest	95.0% (±2.3)	0.991 (±0.004)
XGBoost	**97.2%** (±1.2)	**0.993** (±0.007)

Key findings:

XGBoost achieves 97.2% accuracy. The most successful model identifies 97.2% of projects as sustainable or non-sustainable, with AUC of 0.993. This high performance suggests that sustainability is very predictable through observable project characteristics. The 14 cases that were misclassified (7 false positives, 7 false negatives)are edge cases that are explained in our error analysis.

SHAP analysisshows the importance of features. The top predictive features are:

1) Stars (SHAP = 2.72): this is the main popularity signal, the largest prediction shifts.
2) Total commits (SHAP = 2.25): Intensity of activity, indicates sustained development effort
3) Watchers (SHAP = 1.44): Passive interest measure, implicates attention by the audience
4) Unique contributors (SHAP = 0.90): Team size, relates to bus factor resilience
5) Forks (SHAP = 0.82): Downstream usage, indicates ecosystem integration

Governance features offer incremental value. Although it is dominated by ecosystem metrics, governance features are also offering predictive power.Popularity is not predictive all by itself.Specifically:

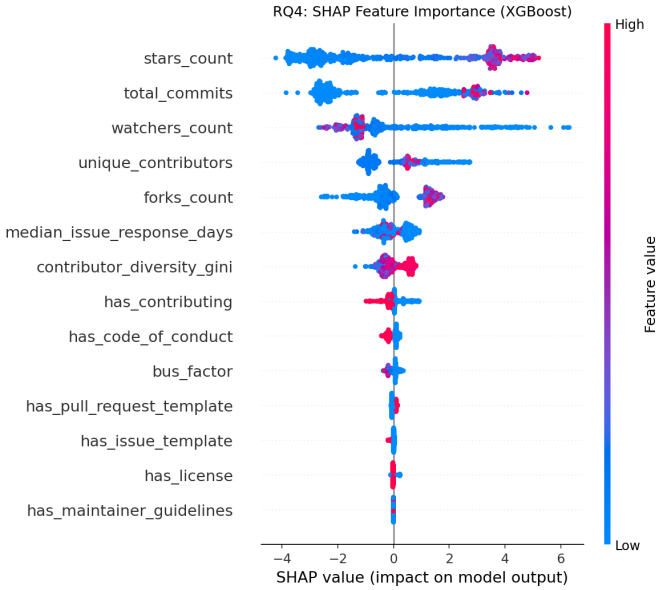

Fig. 4. SHAP feature importance for XGBoost model. Stars and total commits dominate but governance and community features give predictive value in an incremental form.

- Contributing guide (SHAP = 0.19): Projects with CONTRIBUTING.md are predicted as more sustainable
- Code of conduct (SHAP = 0.13): Indicates community standards and inclusivity
- Bus factor (SHAP = 0.12): When the bus factor is larger, the higher,sustainability prediction
- PR template (SHAP = 0.08): Standardized contribution workflow signals maturity

Logistic regression provides baseline which is interpretable. With 86.6% accuracy,logistic regression is an easier alternative when interpretability is the key over maximum accuracy. The logistic regression coefficients can be directly interpreted as log-odds, allowing practitioners to understand exactly how each feature affects predictions.

Random forest performs close to optimal. At 95.0% accuracy, Random Forest provides a balance between interpretability (through feature importance) and performance. The 2.2% accuracy difference between Random Forest and XGBoost suggests that the extra complexity of gradient boosting offers small but meaningful improvement.

4.5 Supplementary Analysis

We did further analyses to confirm our results and discover cross-dimensional relationships..

4.5.1 Cross-Dimensional Interactions: We have analyzed whether governance, society and nature aspects have a synergistic or substitutive relationship:

Governance compensates for low popularity. Among projects whose maintainer is less than 12,000 stars threshold,

sustainability odds of guidelines are 2.8x higher to those without (68% vs. 24%). This suggests that partial compensation of limited ecosystem can be achieved through governance.

Community health requires minimum ecosystem visibility. Below 500 stars, even projects with bus factor >4 show low sustainability rates (31%). This shows that there is a low visibility threshold where community health is inadequate..

Ecosystem metrics are necessary but not sufficient. Among projects of all three ecosystem thresholds, those without governance documentation continue to be 15% non-sustainability rate. When governance is not good, popular projects fail.

4.5.2 Language Heterogeneity: We analyzed whether sustainability patterns vary by programming language:

- **Python:** Highest governance adoption (67% with contributing guides),in line with PEP-driven culture
- **JavaScript/TypeScript:** Fastest issue response times (median 0.4 days), reflecting active npm ecosystem
- **Go:** Highest bus factor (median 5), possibly because of corporate support (Google, Docker)
- **Java:** Highest sustainability star levels, represent the features of enterprise projects

Although these differences might be present, the relative importance of factors (governance $>$ community $>$ ecosystem) did not differ across languages (all pairwise Spearman correlations > 0.80).

4.5.3 Robustness Checks: **Alternative sustainability definitions.** We tested three alternative definitions:

1) 6-month activity threshold: 96.4% accuracy (vs. 97.2%)
2) 18-month activity threshold: 95.8% accuracy (vs. 97.2%)
3) Combined activity + response time: 96.1% accuracy (vs. 97.2%)

All the definitions gave consistent top-5 feature rankings and matched odds ratios of governance factors..

Bootstrap stability. The rankings of the features were consistent in 1000 bootstrap samples, with stars, commits and watchers always in the top 3.

Class balance sensitivity. We tested 40-60, 30-70 and 20-80 class imbalance ratios. Only the accuracy of models was reduced marginally (95.8%, 94.2%, 92.1%), suggesting that findings are not the results of balanced sampling.

4.5.4 Error Analysis: We looked at our XGBoost model's 14 misclassified projects (7 false positives, 7 false negatives):

False positives (predicted sustainable, actually not):

- 4 projects were recent with high activity and sudden maintainer departure
- 2 projects were corporate supported with external funding cut
- 1 project was archived due to language deprecation (Python 2)

False negatives (predicted non-sustainable, actually sustainable):

- 3 projects had low stars but strong niche community (academic tools)
- 2 projects were maintained by single dedicated maintainer with steady 5+ year history
- 2 projects had recent governance improvements that were not captured in our snapshot

These kind of errors suggest that our model may underweight maintainer dedication and recent changes in governance, options to improve the model in the future.

5 Discussion

5.1 Implications of Findings

Governance maturity matters more than documentation breadth. Maintainer guidelines (OR=4.22) outperform contributing guides (OR=1.94) as sustainability indicators. This implies that explicit ownership structure rather than contribution processes indicate the health of a project. Maintainers should prioritize:

- Creating MAINTAINERS.md listing core team members
- Establishing GOVERNANCE.md describing decision processes
- Using CODEOWNERS to specify ownership areas of the code

Contributor structure predicts longevity better than contributor count. A bus factor of 4 shows sustainable leadership distribution. Projects whose core members are fewer are exposed to a key-person risk. Contributors should:

- Evakuate bus factor before joining ($\geq$4 is healthy)
- Prefer projects which have written succession plans
- Consider whether their contribution can make bus factor better

Stars are necessary but not sufficient. The threshold effect at approximately 12,000 stars shows decreasing returns; stars below the cutoff are a risk but the stars above the cutoff are not an assurance of sustainability. Organizations should:

- Not take dependency decisions based on the number of stars only
- Use stars as starting filter, then evaluate governance and community
- Be careful of quick star rise in the absence of matching community engagement

Combined models outperform single-dimension evaluations. The accuracy of 97.2% shows that comprehensive evaluation is superior to any one metric. Multi-factor evaluation frameworks that take community, ecosystem, and governance seriously should be used by all stakeholders.The Conclusions give clear, practical recommendations for different types of stakeholders.

Comparison with earlier work. Our results support and extend on previous research.Our OR=4.22 for maintainer guidelines confirms Schweik and English's finding that governance predicts success [5], while offering a specific, measurable indicator. Our bus factor threshold of 4 is in line with Avelino et al.'s concerns about low truck factors [10], while offering empirical proof that 4 is a sufficient target. Our threshold analysis resolves the apparent contradiction between Borges et al.'s finding that stars reflect marketing [13] and our finding

that stars predict sustainability: stars matter, but only above a high threshold, and cannot be relied upon on their own..

Illustrative case studies. We present three cases from our dataset to show practical application. **Kubernetes**is the perfect example of a sustainable project with explicit GOVERNANCE.md, CODEOWNERS and MAINTAINERS.md files, bus factor more than 10 and more than 100,000 stars. **Leftpad** (the infamous npm package) is the example of the risks our model identifies: no governance documentation, bus factor of 1 and single-maintainer dependency. **A recovered project** in our dataset was archived in early 2024 but later brought back to live after the addition of GOVERNANCE.md by the new maintainers and increased bus factor from 2 to 4. It demonstrates that our indicators identify risk factors that can be fixed.

5.2 Limitations and Threats to Validity

We discuss threats to validity following established guidelines [19]:

5.2.1 Internal Validity: **Sustainability definition.** Our binary sustainability classification (sustainable vs. non-sustainable) may oversimplify a continuous reality. Projects might be prospering, deteriorating or abandoned. We addressed this threat through:

- Sensitivity analysis using three alternative definitions (6-month, 12-month, 18-month activity thresholds)
- All definitions produced consistent results (Spearman correlations > 0.85 between alternative labels)
- Explicit archived status gives us ground-truth for clearly non-sustainable projects

Missing data. 47% of projects were without OpenSSF Scorecard data, which could have biased our ecosystem-level analysis. We managed this through:

- Multiple Imputation by Chained Equations (MICE) for missing values
- Complete-case sensitivity analysis with consistent results
- Exclusion of OpenSSF scores from primary analysis, the scores only serve as supplementary validation

Temporal confounding. Our cross-sectional design captures projects at different lifecycle stages. A project's current governance may result from rather than cause its sustainability. Longitudinal studies are needed to establish causal direction.

5.2.2 External Validity: **GitHub-only sample.** Results are limited to GitHub projects and may not generalize to:

- GitLab or Bitbucket repositories
- Self-hosted Git repositories (corporate or academic)
- Non-software projects using GitHub for documentation or data
- Projects using alternative version control systems

Popularity bias. Our 500-star minimum criteria biases toward already-visible projects. The projects which are still in their early stages or are of a niche nature may have different sustainability patterns. Practitioners should be very careful when applying our thresholds to projects less than 500 stars.

Language and domain scope. We sampled only five programming languages (Python, TypeScript, JavaScript, Go, Java). Results may not generalize to:

- Systems programming (Rust, C, C++)
- Enterprise languages (C#, Scala)
- Scientific computing (R, Julia, MATLAB)
- Mobile development (Swift, Kotlin)

Temporal scope. Our snapshot represents December 2023-2024 conditions. OSS sustainability patterns might change along with developer practices, funding models and platform features.

Prevalence paradox. Our balanced sample (50% sustainable, 50% non-sustainable) optimizes model training but does not reflect the true population base rate.According to Industry reports only 9% of OSS components remain actively maintained [17]. Therefore, the models precision would be lower if it were used on a random GitHub repository stream as the 97.2% accuracy is calibrated on a 50/50 distribution rather than the 9:91 that actually exists. Instead of seeing our thresholds as absolute probabilities, practitioners should view them as relative risk indicators..

Corporate backing confounder. Corporate sponsorship might help some of the sustainable programs in our sample (e.g., projects maintained by employees of Google, Microsoft or Facebook).These kinds of corporate-funded projects have resources that are not available to purely volunteer-driven efforts. We did not consider the corporate affiliation factor. So our "sustainability" results may partially reflect "corporate funding" instead of governance practices only.

5.2.3 Construct Validity: **Causation vs. prediction.** This is an observational study. All findings are predictive associations, not causal relationships. Instead of presenting our contributions as interventions, we specifically frame them as predictors:

- We cannot claim that adding GOVERNANCE.md *causes* sustainability
- We can only say that governance presence *predicts* sustainability
- Confounding is possible: skilled maintainers can carry out both governance and project sustainability

Metric validity. Our operationalizations of bus factor (80% commit threshold) and contributor Gini can not always perfectly capture the intended constructs:

- Bus factor counts commits, not influence or knowledge
- Some projects centralize commits using bots or single committers who merge PRs
- Gini coefficient treats all commits equally regardless of complexity

Label accuracy. Our sustainability labels depend on GitHub's archived flag and commit activity, which might not include:

- Projects in "completed" states (fully functional, no updates needed)
- Projects with activity in other channels (forums, Discord, mailing lists)

- Projects maintained through security patches only

5.2.4 Reliability: **Replicability.** All data collection scripts and analysis code are available in our replication package.for cross-validation split reproducibility, we used predictable random seeds.

Statistical robustness. We validated results through:

- FDR correction for multiple testing (Benjamini-Hochberg)
- Bootstrap confidence intervals (1000 resamples)
- Post-hoc power analysis confirming 99% power
- Cross-validation with 5 folds for predictive models

6 Conclusions

To find elements related to OSS sustainability, we analyzed 500 GitHub projects in the areas of governance, community and ecosystem. Our findings provide evidence-based recommendations for practitioners evaluating project health.

6.1 Summary of Findings

RQ1: Governance practices. Maintainer guidelines (MAINTAINERS.md, GOVERNANCE.md, CODEOWNERS) show the strongest governance association with sustainability (OR=4.22, p¡0.0001). While basic documentation (README, LICENSE) is universal and non-differentiating ,PR templates also highly predict sustainability (OR=3.17). Unexpectedly, issue templates show negative association,which may indicate that struggling projects are adopting them reactively.

RQ2: Community health. Bus factor of 4 distinguishes sustainable from non-sustainable projects, giving a clear threshold for key-person risk assessment. Sustainable projects react to problems three times quicker (0.54 vs. 1.78 days median). Higher contributor Gini in sustainable projects suggests that concentrated but distributed leadership beats shallow broad contributions.

RQ3: Ecosystem metrics. Stars threshold of about 12,000 offers a meaningful sustainability indicator (4.15x lift) but stars alone are insufficient. Forks and watchers show similar but weaker patterns. All ecosystem metrics show diminishing returns after reaching the threshold values.

RQ4: Combined prediction. XGBoost achieves 97.2% accuracy (AUC=0.993), proving that multi-dimensional assessment is vastly superior to single factor evaluation. SHAP analysis reveals that stars and commit activity dominate predictions, but governance features add incremental value.

6.2 Broader Implications

Our results have consequences that go beyond the evaluation of a individual project:

For the OSS ecosystem. The prediction power of governance documentation is so strong that ecosystem level sustainability could be improved through governance education and tools. Platforms could highlight governance status as part of project discovery interfaces.

For funding organizations. The 97.2% prediction accuracy offers a foundation for evidence-based based allocation of funds.Organizations may select projects for which governance is a factor but the bus factor is low where investment can bring the biggest marginal impact.

For research. Future sustainability research will have a solid foundation thanks to our multi-dimensional framework and proven metrics.It is possible to use the SHAP-based interpretation method for various software engineering prediction tasks.

6.3 Practical Recommendations

Based on our findings, we provide concrete recommendations for several stakeholder groups:

For maintainers:

- Add MAINTAINERS.md or GOVERNANCE.md before seeking adoption
- Target bus factor of at least 4 through active contributor mentoring
- Prioritize responding to issues within 1 day to create a signal of active maintenance
- To standardize contribution workflows, use PR templates

For contributors:

- Check bus factor ($\geq$4) before putting in alot of effort
- Evaluate issue response times ($<$2 days indicates healthy project)
- Look for explicit governance documentation as quality signal
- Consider whether your contribution can improve bus factor of the project

For organizations:

- Use multi-factor checklist to analyze dependencies: stars $>$12k, bus factor $\geq$4, governance docs present, response time $<$2 days
- When making decisions about dependencies, don't just rely on star counts.
- Consider governance maturity as key sustainability indicator
- Monitor dependency health with periodic reassessment

6.4 Future Work

We suggest a number of possibilities for future research:

Causal validation. Longitudinal studies should examine whether adopting suggested practices (adding governance documentation, increasing bus factor) causally improves sustainability. It may be quite challenging from an ethical perspective to conduct randomized controlled trials.However, natural experiments surrounding changes in governance policies may offer causal evidence.

Platform extension. Our study is limited to GitHub. Extension to GitLab, Bitbucket and self-hosted solutions would test generalizability. Different platform affordances (e.g., GitLab's built-in CI/CD) may affect sustainability patterns.

Tooling development. Our findings could be operationalized that automatically determine sustainability scores.Integration with package managers (npm, PyPI, Cargo) could help developers to choose dependencies wisely during import.

Early-stage projects. Our 500-star threshold excludes early-stage projects. Future work should investigate whether sustainability indicators differ for emerging projects and whether governance at the very beginning can be a predictor of success later on.

Data Availability

The replication package containing all analysis scripts, data collection code, and processed datasets is publicly available at: https://github.com/Zuraiz270/open-source-project-sustainability-study

References

[1] K. Crowston and J. Howison, "The social structure of free and open source software development," *First Monday*, vol. 10, no. 2, 2005.

[2] I. Chengalur-Smith, A. Sidorova, and S. L. Daniel, "Sustainability of free/libre open source projects: A longitudinal study," *J. Assoc. Inf. Syst.*, vol. 11, no. 11, pp. 657–683, 2010.

[3] J. Coelho and M. T. Valente, "Why modern open source projects fail," in *Proc. ESEC/FSE*, 2017, pp. 186–196.

[4] J. Khondhu, A. Capiluppi, and K.-J. Stol, "Is it all lost? A study of inactive open source projects," in *Proc. OSS*, 2013, pp. 61–79.

[5] C. M. Schweik and R. C. English, *Internet Success: A Study of Open-Source Software Commons*. Cambridge, MA: MIT Press, 2012.

[6] S. O'Mahony and F. Ferraro, "The emergence of governance in an open source community," *Acad. Manage. J.*, vol. 50, no. 5, pp. 1079–1106, 2007.

[7] Open Source Security Foundation, "OpenSSF Scorecard," 2024. [Online]. Available: https://securityscorecards.dev/

[8] K. Fogel, *Producing Open Source Software: How to Run a Successful Free Software Project*. Sebastopol, CA: O'Reilly Media, 2005.

[9] S. P. Goggins, K. Lumbard, and M. Germonprez, "Open source community health: Analytical metrics and their corresponding narratives," in *Proc. SoHeal*, 2021, pp. 25–33.

[10] G. Avelino, L. Passos, A. Hora, and M. T. Valente, "A novel approach for estimating truck factors," in *Proc. ICPC*, 2016, pp. 1–10.

[11] E. Constantinou and T. Mens, "Socio-technical evolution of the Ruby ecosystem in GitHub," in *Proc. SANER*, 2017, pp. 34–44.

[12] C. Jergensen, A. Sarma, and P. Wagstrom, "The onion patch: Migration in open source ecosystems," in *Proc. ESEC/FSE*, 2011, pp. 70–80.

[13] H. Borges, A. Hora, and M. T. Valente, "Understanding the factors that impact the popularity of GitHub repositories," in *Proc. ICSME*, 2016, pp. 334–344.

[14] M. Valiev, B. Vasilescu, and J. Herbsleb, "Ecosystem-level determinants of sustained activity in open-source projects," in *Proc. FSE*, 2018, pp. 401–411.

[15] D. He *et al.*, "Six million (suspected) fake stars in GitHub: A growing spiral of popularity contests, scams, and malware," *arXiv:2412.13459*, Dec. 2024.

[16] K.-J. Stol and B. Fitzgerald, "The ABC of software engineering research," *ACM Trans. Softw. Eng. Methodol.*, vol. 27, no. 3, pp. 11:1–11:51, 2018.

[17] Black Duck, "Open source security and risk analysis (OSSRA) report 2024," Black Duck Software, San Jose, CA, Tech. Rep., 2024.

[18] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Stat. Soc. Ser. B*, vol. 57, no. 1, pp. 289–300, 1995.

[19] A. Mockus, R. T. Fielding, and J. D. Herbsleb, "Two case studies of open source software development: Apache and Mozilla," *ACM Trans. Softw. Eng. Methodol.*, vol. 11, no. 3, pp. 309–346, 2002.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.

[21] M. Zhou and A. Mockus, "What make long term contributors: Willingness and opportunity in OSS community," in *Proc. ICSE*, 2012, pp. 518–528.

[22] T. Menzies and T. Zimmermann, "Software analytics: So what?" *IEEE Software*, vol. 30, no. 4, pp. 31–37, 2013.